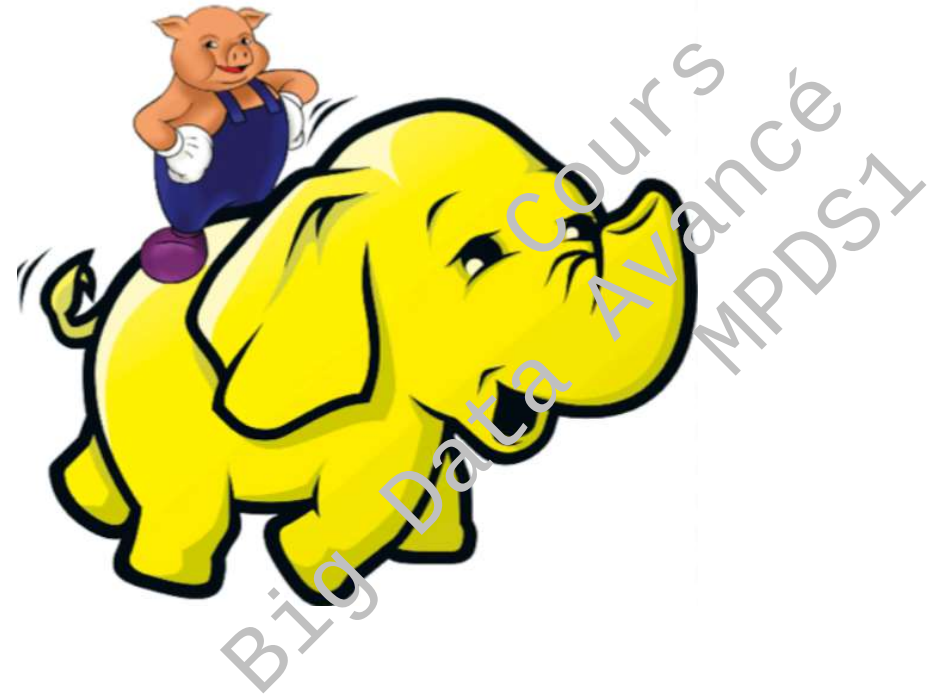
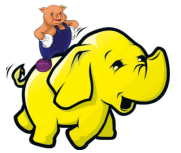


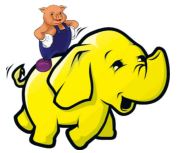
CH PIG

- ☐ Présentation
- ☐ Pig vs MapReduce
- ☐ Architecture & interaction
- ☐ Type de données, Schéma
- ☐ Pig Latin
- ☐ Exercices



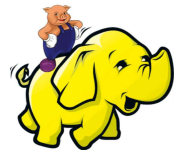


- **C'est quoi « Pig »**
- Pig est un langage de script de haut niveau utilisant Apache Hadoop comme plateforme logicielle.
- Pig permet aux utilisateurs d'écrire des programmes manipulant des données complexes sans avoir forcément connaître Java.
- Pig est un langage développé par l'équipe de Yahoo afin de surmonter la complexité de programmer en Java selon le paradigme MapReduce.
- Pig est créé pour programmer rapidement des programmes de manipulation des données.
 - une ligne Pig = 20 lignes en Java
 - temps de développement (Pig) = 1/16 temps de développement (Java)
- Pig est un langage de manipulation des données **impératif** (contraire au langage SQL)

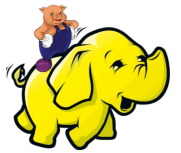


- Pig travaille avec des données provenant de nombreuses sources, y compris des données structurées et non structurées, et stocke les résultats dans HDFS : Hadoop Data File System.
- Son nom ? « Pig » est inspiré du l'animal domestique « cochon » qui avale tout type de nourritures. Par analogie, Apache Hadoop Pig traite tout types de données (structurées, semi-structurées, non structurées)
- Les scripts pig sont traduits en une série de tâches Mapreduce qui sont exécutées sur le cluster Apache Hadoop.
- Pig est différent des langages de programmation procédurale et/ou orienté objet. C'est un langage de traitement de flux de données
- Pig est **un langage de requêtes et de traitement de données.**
- Pig est similaire à SQL, il fournit des fonctions « built-in » pour traiter les flux des données (jointure, concaténation, ...).
- Pig a ces **propres structures de données** (atom, tuples, bag, map)

PIG Vs MapReduce

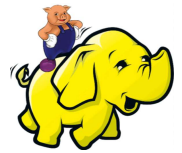


Pig	MapReduce
Apache Pig est un langage de flux de données.	Mapreduce est un paradigme de traitement de données.
C'est un langage de haut niveau.	Mapreduce est bas niveau et rigide.
L'exécution d'une opération de jointure « Join » avec Apache Pig est assez simple.	Dans Mapreduce, il est assez difficile d'effectuer une opération de jonction entre données.
Tout programmeur novice avec une connaissance de base de SQL peut travailler facilement avec Apache Pig.	Une connaissance avancée en Java est requise pour travailler avec Mapreduce.
Apache Pig utilise une approche multi-requête, ce qui réduit considérablement la longueur des codes.	Mapreduce nécessitera presque 20 fois plus de lignes pour effectuer la même tâche.

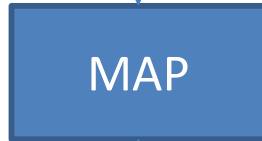


- Exercice : comptage des objets d'un enfant
 - un enfant joue avec son peluche Hadoop
 - un enfant joue avec son ballon rouge
 - un enfant dessine avec son stylo sur les murs
 - un enfant monte sur sa bicyclette

PIG Vs MapReduce

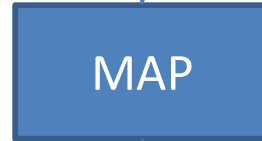


un enfant joue avec
son peluche Hadoop



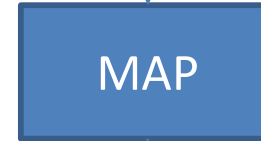
Un : 1
Enfant : 1
Joue : 1
Avec :
son : 1
Peluche : 1
Hadoop : 1

un enfant joue avec
son ballon rouge



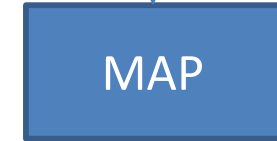
un : 1
Enfant : 1
Joue : 1
Avec : 1
Son : 1
Ballon : 1
Rouge : 1

un enfant dessine avec
son stylo sur les murs

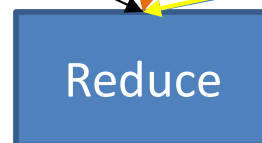


un : 1
Enfant : 1
Dessine : 1
Avec : 1
son : 1
Stylo : 1
Sur : 1
les : 1
Murs : 1

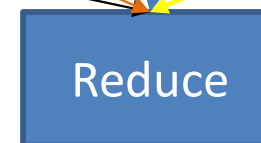
un enfant monte sur
sa bicyclette



Un : 1
Enfant : 1
Monte : 1
sur : 1
Sa : 1
Bicyclette : 1

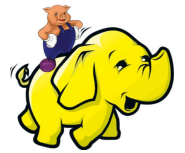


Un : 4
Enfant : 4
Joue : 2
Avec : 3
son : 3
Hadoop : 1
...



ballon : 1
Peluche : 1
Stylo : 1
Bicyclette : 1
Rouge : 1
...

PIG Vs MapReduce



-- Load : télécharger le fichier 'enfant' et nommer chaque champ 'line'
input = load 'enfant' as (line);

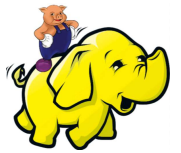
-- TOKENIZE : segmenter la ligne en champs isolés
-- flatten : créer un enregistrement 'mots' des champs isolés générés par TOKENIZE. Les champs isolés sont appelés 'mot'
mots = foreach input generate flatten(TOKENIZE(line)) as (mot);

-- group : grouper les mots
grp = group mots by mot;

-- Count : comptage
cntd = foreach grp generate group, COUNT(mots);

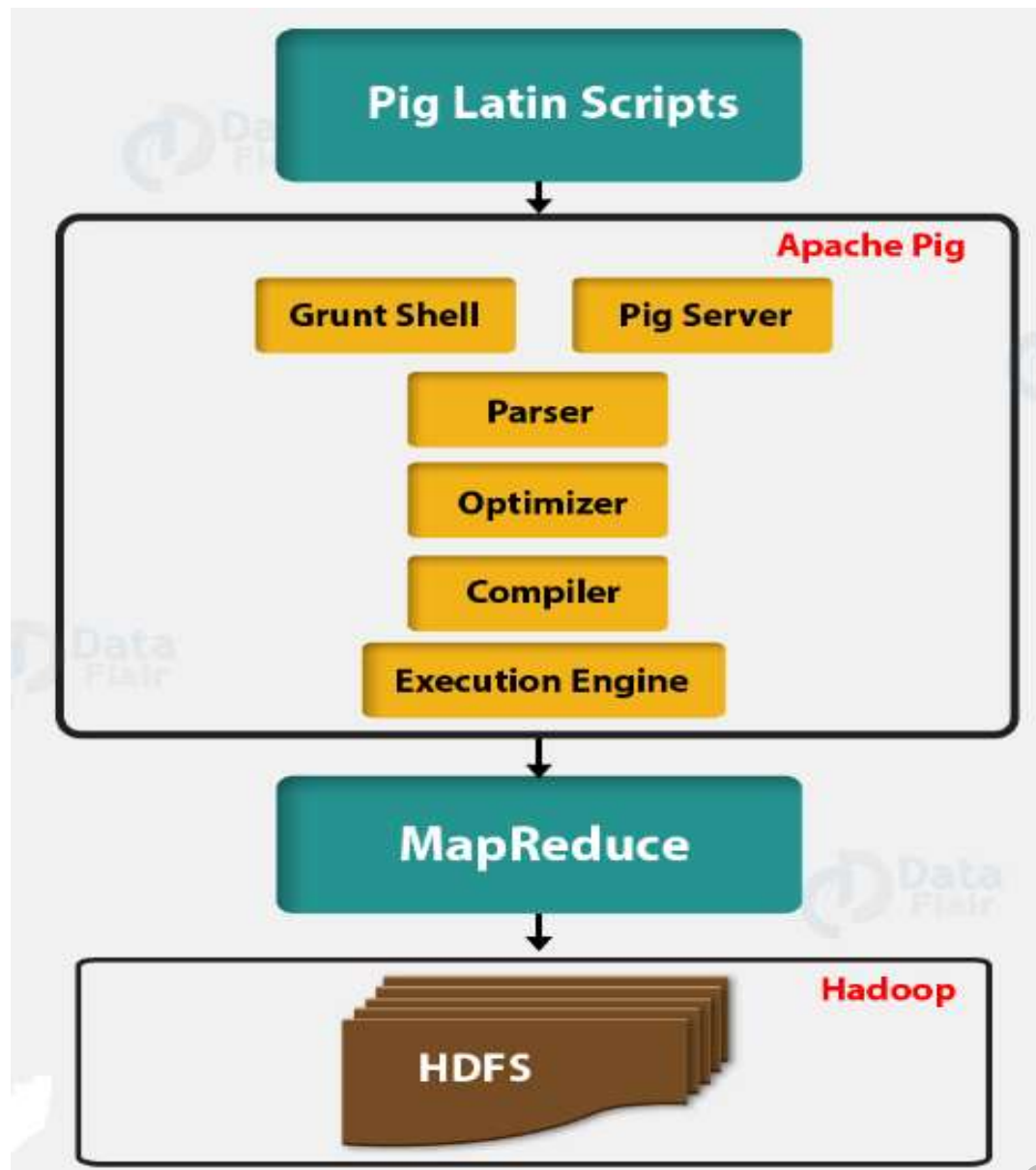
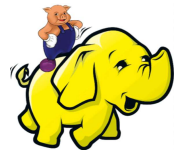
-- dump : afficher le résultat.
dump cntd;

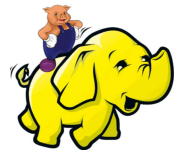
➡ **Seulement 4 lignes de Pig remplacent plus de 80 lignes Mapreduce**



- Pig Latin est un langage qui ressemble à SQL. Il y a plusieurs mots clés en commun (JOIN , ORDER , LIMIT . . .) mais leurs principes sont différents :
- En **SQL**, on construit des requêtes qui **décrivent les données à obtenir**. On ne sait pas comment le moteur SQL va faire pour calculer le résultat. On sait seulement qu'en interne, la requête va être décomposée en boucles et en comparaisons sur les données et en utilisant au mieux les index.
- En **Pig Latin**, on construit des programmes qui contiennent des instructions. On **décrit exactement comment** le résultat doit être obtenu, quels calculs doivent être faits et dans quel ordre.
- Pig a été conçu pour les données incertaines de Hadoop, tandis que SQL tourne sur des SGBD parfaitement sains.

PIG : architecture

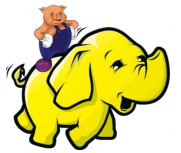




- Pig est composé essentiellement de deux modules
 - **Pig Latin**: langage procédural de haut niveau
 - **Pig Engine** : parse, optimise, compile et exécute automatiquement les scripts PigLatin comme une série de jobs MapReduce au sein d'un cluster Hadoop.

- **Parser** (vérification)
 - « Parser » vérifie essentiellement la syntaxe du script, fait la vérification de type, et d'autres vérifications diverses.
 - La sortie de « Parser » est un **DAG (directed acyclic graph)** qui représente les énoncés Pig Latin ainsi que les opérateurs logiques.
 - Les opérateurs logiques du script sont représentés par des nœuds et les flux de données sont représentés par des arrêtes dans DAG (le plan logique)

- **Optimisateur**
 - Par la suite, le plan logique (DAG) est passé à l'optimiseur logique. Il effectue des optimisations logiques



- **Compilateur**

Ensuite, le compilateur compile le plan logique optimisé dans une série de jobs MapReduce.

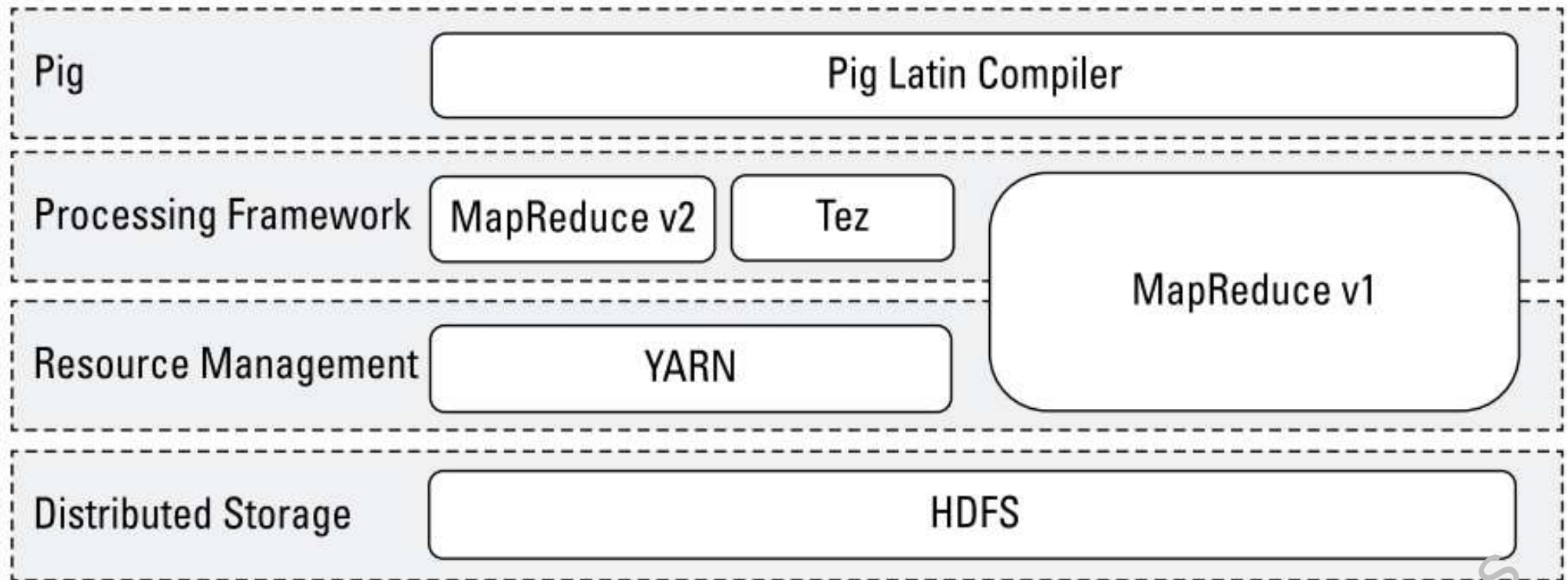
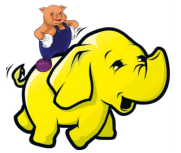
- **Moteur d'exécution**

Finalement, tous les jobs MapReduce sont soumis à Hadoop dans un ordre trié.

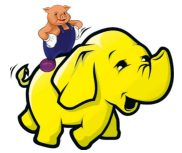
- **Remarque :**

Pig s'installe sur **la machine local** de l'utilisateur. Il ne nécessite aucune installation sur le cluster Hadoop

PIG : architecture



- Les programmes Pig peuvent fonctionner sur Mapreduce v1 ou Mapreduce v2 sans aucun changement du code.
- Les scripts Pig peuvent fonctionner en utilisant l'**API** Tez. Apache Tez fournit un cadre d'exécution plus performant que Mapreduce.



■ Modes d'exécution Apache Pig

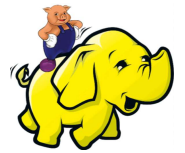
On peut exécuter Apache Pig dans deux modes, à savoir le **mode local** et le **mode HDFS**.

- Mode local

Dans ce mode, tous les fichiers sont installés et exécutés à partir de du hôte local et le système de fichiers local. Il n'y a pas besoin de Hadoop ou HDFS. Ce mode est généralement utilisé à des fins de test.

- Mapreduce Mode

Mapreduce est le mode où on charge ou on traite les données qui existent dans le système de fichiers Hadoop (HDFS). Dans ce mode, chaque fois qu'on exécute des instructions Pig Latin pour traiter les données, une tâche Mapreduce est invoquée dans le back-end pour effectuer une opération particulière sur les données qui existent dans le HDFS.



- **Mode interactif (Grunt shell)**

Il s'agit d'interagir avec le shell Pig. Dans ce mode, on peut lancer des traitements MapReduce ou interagir avec HDFS.

exécution :

```
$ pig -x local ou $ pig -x mapreduce  
grunt>
```

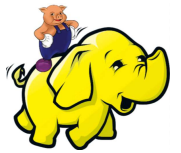
- **Batch Mode (Script)**

On peut exécuter Apache Pig en mode Batch en écrivant le script Pig Latin dans un seul fichier avec l'extension **.pig**

```
$pig -x local script.pig ou $pig -x mapreduce script.pig
```

- **Mode Intégré (UDF)**

Apache Pig fournit la possibilité de définir des fonctions (User Defined Functions) dans d'autres langages de programmation tels que Java et de les utiliser comme des API.

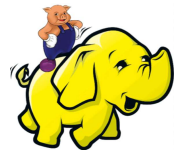


- Les types de données de Pig peuvent être divisés en deux catégories
 - **Atom** : qui contiennent une seule valeur
 - **Complexe** : qui contiennent d'autres types.

- **Atom**

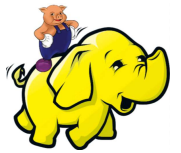
Atom représente les types de données simples qu'on trouve presque dans tous les langages de programmation.

- Int : représente les entiers. codé sur 4 bytes signés = 32bits.
- Long : représente les entiers de type long. codé sur 8 byte = 64 bits signés.
- Float : désigne les données de type réel. Il est codé sur 4 bytes = 32 bits.
- Double : désigne les données de type réels (double précision). Il est codé sur 8 bytes = 64 bits.
- Chararray : permet de représenter les string ou un tableau de caractère.
- Bytearray, Boolean, BigInteger, BigDecimal



▪ *Tuple*

- C'est une collection ordonnée.
- Le tuple est composé de plusieurs champs, chaque champ peut être de n'importe quel type.
- Les éléments d'un tuple ne sont pas forcément de même type.
- Il est possible d'accéder à un champs à travers sa position.
- Pig permet d'associer **un schéma à un tuple** pour **décrire le type** de chaque champ et lui **donner un nom**. En l'occurrence, il est possible **d'accéder à un champ par son nom**.
- Exemple
 ('MPDS1', 10, 4)
 (2,5,6)



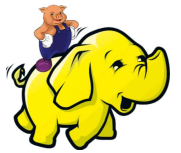
■ **Bag**

- Une collection de tuple non ordonnée (pas notion de position)
- Exemple
`{('MPDS', 1), ('MR', 'Info', 2), ('DepInfo', 'FSB')}`

■ **Map**

- C'est dictionnaire, composé d'une clé et une valeur.
- La clé est de type **chararray**, elle est utilisée comme index pour trouver la valeur associée.
- La valeur est considéré comme un «**bytearray**».
- Exemple
`[ 'nom'# 'Kamel', 'age'# 25]`

Types de données : type Complexe

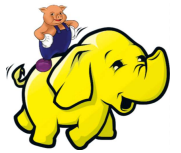


- Extraction des champs d'un type complexe : il faut utiliser un opérateur de projection selon le type des données
- Tuple → opérateur : **(.)** le point

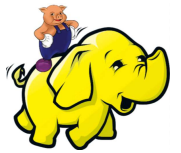
```
A = load 'input' as (t:tuple(x:int, y:int));
B = foreach A generate t.x, t.$1;
```
- Map → opérateur : **(#)**

```
bball = load 'baseball' as (name:chararray, team:chararray,
position:bag{t:(p:chararray)}, bat:map[]);
avg = foreach bball generate bat#'batting_average';
```
- Bag → opérateur : **(.) MAIS**
on crée un Bag et non pas un scalaire

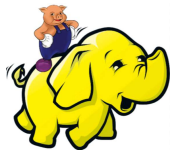
```
A = load 'input' as (b:bag{t:(x:int, y:int)});
B = foreach A generate b.x;
-- B est un Bag contenant uniquement les valeurs (x)
```



- Pig inclut le concept d'un élément de données **Null**.
- Les données de tout type peuvent être null.
- le concept de null dans Pig est le même que dans SQL, ce qui est complètement différent du concept de null en C, Java, Python, etc.
- Dans Pig un élément de données null signifie que la valeur est **inconnue**. Ceci peut être parce que les données sont manquantes, une erreur s'est produite dans leur traitement, etc.



- **Les schémas permettent d'associer les noms et les types des champs.** Ils sont facultatifs mais recommandés, car ils permettent le contrôle des erreurs ainsi qu'une meilleure optimisation d'exécution du code.
- La façon la plus simple de communiquer le schéma de données à Pig est de dire explicitement à Pig le type des données lors de leur chargement
- Exemple
etudiants = **load** 'MPDS1' **as** (nom:**chararray**, prénom:**chararray**, age:**int**);
Pig s'attend à ce que les données aient 3 champs. **S'il y en a plus**, il va **tronquer** les champs supplémentaires. **S'il y en a moins**, Il va augmenter avec des **Null**.
- Il est également possible de spécifier le schéma sans donner de types de données explicites. Dans ce cas, le type de données est supposé être **bytearray**
etudiants = **load** 'MPDS1' **as** (nom, prénom, age);



- Une **relation** est un **bag** de **Tuple**

exemple

```
etudiants = load 'MPDS1' as (nom:chararray, prénom:chararray, age:int);
```

- Pig référence un champs par sa position ou son nom.

- **accès par position** : utilisation de l'opérateur **\$** suivi par le numéro de la position. la position **n** est repérée par **\$(n-1)**.

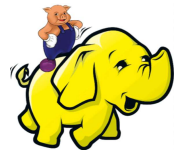
- **accès par nom** : le nom désigne l'identificateur du champs.

- Exemple

```
X = FOREACH etudiants GENERATE prénom, $1;  
DUMP X
```

Accès par clé

Accès par position

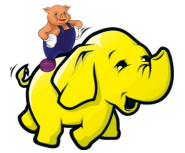


- Pig Latin est un **langage de flux de données**. Chaque étape de traitement résulte en **un nouvel ensemble de données**, ou **relation**.
- Exemple
 - `A = load 'data'`
 - `A` : le nom de la relation qui résulte du chargement des données de l'ensemble de données.
 - `A` : nom de relation est appelé un alias.
 - Les noms de relation ressemblent à des variables, mais ils ne le sont pas.

- Exemple

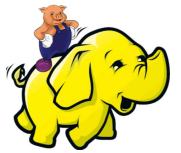
Il semble ici comme si on réaffecte `A`, mais vraiment on crée des nouvelles relations appelées `A`, et on perd la trace des anciennes relations appelées `A`.

```
A = load 'articles' (catégorie, type, date, prix);  
A = filter A by prix > 100;  
A = foreach A generate UPPER(type);
```



- *Remarque : Sensibilité à la casse*
 - Pig Latin n'est pas sensible à la casse : **LOAD** et **load** sont identiques
 - Les variable sont sensible à la casse : **TYPE** est différent de **type**
- Chargement des données : **load**
 - le chargement des données se fait par l'instruction **load**
 - par défaut, Pig charge les données à partir de HDFS et selon un path prédéfinie.
 - il est aussi possible d'effectuer un chargement des données via un autre système de stockage tel que HBase.

```
A = load 'MPDS1' using HBaseStorage(); --Hbase
A = load 'MPDS1' using PigStorage(','); --HDFS (par défaut), délimiteur (,)
A = load 'MPDS1' as (nom:chararray, prénom:chararray, age:int);
/*
* Puisque la méthode n'est pas déclarée → PigStorage →HDFS
* As : pour spécifier le schéma
*/
```



- Stockage : **store**
 - Par défaut, Pig stocke les données sur HDFS dans un fichier en faisant appel à PigStorage.
 - Il est possible de préciser d'autre fonction comme HBaseStorage.
 - L'écriture des résultats de traitement nécessite le chemin relatif ou le chemin absolu du fichier.

```
store res into '/data/resultatTraitement'; -- chemin relatif
```

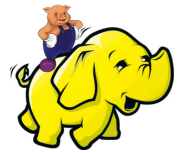
```
store res into hdfs://data.process.com/data/resultatTraitement; --chemin absolu
```

```
store res into '/data/resultatTraitement' using PigStorage(); --avec précision HDFS
```

- Affichage : **dump**
 - Afficher de données l'écran

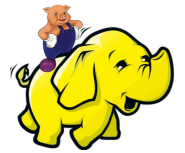
```
dump res
```

Pig Latin : opérations arithmétiques



Operation	Symbol	
addition	+	
subtraction	-	
multiplication	*	
division	/	
modulo	%	Le reste de division (int, long)
bincond	? :	Condition booléenne Doit être entre ().
case	CASE WHEN THEN ELSE END	Exécution conditionnée

Pig Latin : opérations arithmétiques



Exemple : Soit A

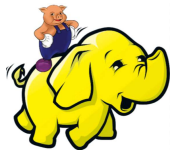
```
A = LOAD 'data' AS (f1:int, f2:int);
DUMP A;
(10,1)
(10,3)
(10,6)

X = FOREACH A GENERATE f2, (
    CASE f2 % 2
        WHEN 0 THEN 'even'
        WHEN 1 THEN 'odd'
    END
);

DUMP X;
(1,odd)
(3,odd)
(6,even)
```

Pour plus d'information voir le site officiel : Pig

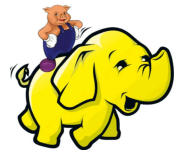
<http://pig.apache.org/docs/latest/basic.html#arithmetic-ops>



- Pig offre plusieurs outils de transformation de données : **le tri, le groupement, la jointure, le filtrage...**
- **Foreach**
 - Applique un ensemble d'opérations à chaque enregistrement dans le pipeline de données (d'où le nom foreach)

Ce code télécharge les données, mais supprime tous les champs sauf l'utilisateur et l'identifiant de l'enregistrement

```
A = load 'input' as (user:chararray, id:long, address:chararray,  
phone:chararray, preferences:map[]);  
B = foreach A generate user, id;
```

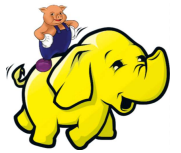


■ Filter

- Sélection d'un tuple d'une relation selon une condition
- Syntaxe : `alias = FILTER alias BY expression;`

```
A = LOAD 'data' AS (f1:int, f2:int);
DUMP A;
(10,1)
(10,3)
(10,6)

B = filter A by f2 > 2
Dump B
(10,3)
(10,6)
```



- **group**
- Grouper les données

Détails des étudiants

Kamel, 22, bizerte, 10,61

Olfa, 23, tunis, 11,96

Karim, 21, kairouan, 13,02

Jihene, 21, sousse, 11,05

```
A = LOAD 'hdfs://localhost:9000/pig_data/liste_etudiant.txt' USING  
PigStorage(',') as (nom:chararray, age:int, ville:chararray, moyenne:float);
```

```
group_A1 = GROUP A by age;
```

```
Dump group_A1
```

```
(21, {(Karim, 21, kairouan, 13,02), (Jihene, 21, sousse, 11,05)})
```

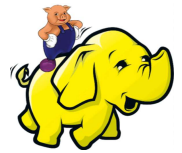
```
(22, {(Kamel, 22, bizerte, 10,61)})
```

```
(23, {(Olfa, 23, tunis, 11,96)})
```

Voir aussi

```
group_A2 = GROUP A by (age, ville);
```

Pig Latin : opérateurs relationnels



▪ join

- Jointure des données
- Syntaxe : Relation3 = JOIN Relation1 BY key, Relation2 BY key ;

D1: Détails des étudiants

Kamel, 22, bizerte, 10.61
Olfa, 23, tunis, 11.96
Anouar, 29, Bizerte, 13.02
Jihene, 21, sousse, 11.05

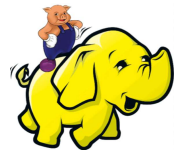
D2: Détails des étudiants

Kamel, football
Olfa, natation
Anouar, **spéléologie**
Jihene, handball

```
D1 = LOAD 'hdfs://localhost:9000/pig_data/D1.txt' USING PigStorage(',') as  
(nom:chararray, age:int, ville:chararray, moyenne:float);  
D2 = LOAD 'hdfs://localhost:9000/pig_data/D2.txt' USING PigStorage(',') as  
(nom:chararray, sport:chararray);
```

```
D1D2 = join D1 by nom, D2 by nom ;  
Dump D1D2  
(Kamel, 22, bizerte, 10.61,football)  
(Olfa, 23, tunis, 11.96, natation)  
(Anouar, 29, Bizerte, 13.02, spéléologie)  
(Jihene, 21, sousse, 11.05, handball)
```

Pig Latin : opérateurs relationnels



■ union

- concaténation des données
- Syntaxe : Relation3 = union Relation1, Relation2 ;

D1: Détails des étudiants

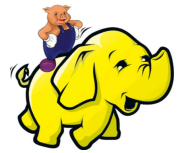
Kamel, 22, bizerte, 10.61
Olfa, 23, tunis, 11.96
Karim, 21, kairouan, 13.02
Jihene, 21, sousse, 11.05

D2: Détails des étudiants

Hanene, 24, bizerte, 10.32
Moez, 21, tunis, 11.15

```
D1D2 = union D1, D2;  
Dump D1D2  
(Kamel, 22, bizerte, 10.61)  
(Olfa, 23, tunis, 11.96)  
(Karim, 21, kairouan, 13.02)  
(Jihene, 21, sousse, 11.05)  
(Hanene, 24, bizerte, 10.32)  
(Moez, 21, tunis, 11.15)
```

Pig Latin : opérateurs relationnels



■ limit

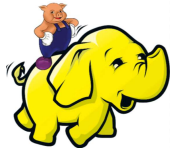
- Limite le nombre de tuple en sortie
- Syntaxe : alias = LIMIT alias n;

```
D1D2 = union D1, D2;  
D1D2_limit = limit D1D2 3  
Dump D1D2_limit  
(Kamel, 22, bizerte, 10.61, football)  
(Olfa, 23, tunis, 11.96, natation)  
(Karim, 21, kairouan, 13.02, spéléologie)
```

■ split

- Segmenter une relation en deux ou plusieurs relations
- Syntaxe : SPLIT alias INTO alias IF expression, alias IF expression;

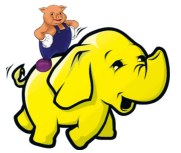
```
D1D2 = union D1, D2;  
Split D1D2 into X if age >= 23, Y if moyenne >= 11.50  
Dump X  
(Olfa, 23, tunis, 11.96, natation)  
(Hanene, 24, bizerte, 10.32)  
Dump Y  
(Olfa, 23, tunis, 11.96, natation)  
(Karim, 21, kairouan, 13.02, spéléologie)
```



Pour le reste des relations, consulter le site officiel de
Apache Pig

<http://pig.apache.org/docs/latest/basic.html#join-inner>

Cours
Big Data Avancé
MPDS1



■ Exercice

On vous donne un fichier 'Magasin_Aziza' dont le contenu est le suivant :

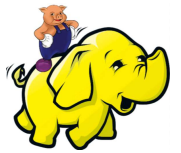
Bizerte, 1897, 5254, 251, 0.210, 8, 20

Jarzouna, 2541, 7863, 365, 0.655, 9, 22

Bizerte_Corniche, 563, 2988, 67, 0.150, 8, 21

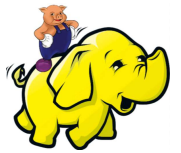
```
A = load 'Magasin_Aziza_Bizerte.txt' as (ville, NBarticles,  
ChiffreAffaire, prixhigh, prixlow, ouverture, fermeture);  
diff1 = foreach A generate prixhigh - prixlow;  
diff2 = foreach A generate $3 - $4;
```

- 1) Analyser le code et commenter sur les valeurs de diff1 et diff2
- 2) Ecrire un code en Pig Latin qui permet de trier les magasins selon leurs chiffres d'affaire puis afficher celle dont le chiffre d'affaire est le plus élevé.
- 3) Afficher les magasins dont le nombre d'articles vendus dépasse 2000 articles
- 4) Afficher la durée d'ouverture de chaque magasin



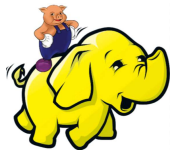
■ Exercice :

- Soit un fichier nommé 'patients.txt' contenant 4 colonnes séparées par (:) :
identifiant: nom: age: maladie
- On vous demande d'écrire un programme en Pig Latin qui fait afficher les 10 patient dont l'âge est compris entre 15 et 25 dans un ordre décroissant

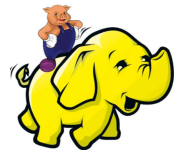


```
public class ObjectCountMapper extends Mapper<LongWritable, Text, Text,
IntWritable> {
    private final static IntWritable one = new IntWritable(1);
    private Text object = new Text();

    public void map(LongWritable key, Text value, Context context)
throws IOException, InterruptedException {
        String line = value.toString();
        String[] parts = line.split(" ");
        object.set(parts[3]);
        context.write(object, one);
    }
}
```

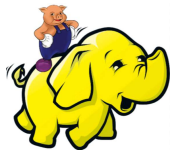


```
public class ObjectCountReducer extends Reducer<Text, IntWritable,
Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context
context) throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
}
```



```
public static void main(String[] args) throws Exception {  
    Configuration conf = new Configuration();  
    Job job = Job.getInstance(conf, "Object Count");  
    job.setJarByClass(ObjectCount.class);  
    job.setMapperClass(ObjectCountMapper.class);  
    job.setCombinerClass(ObjectCountReducer.class);  
    job.setReducerClass(ObjectCountReducer.class);  
    job.setOutputKeyClass(Text.class);  
    job.setOutputValueClass(IntWritable.class);  
    FileInputFormat.addInputPath(job, new Path(args[0]));  
    FileOutputFormat.setOutputPath(job, new Path(args[1]));  
    System.exit(job.waitForCompletion(true) ? 0 : 1);  
}
```

Annexe : exemple de référencement des champs



```
cat data.log;

(jim, 6, 8,9) (Rach, 7,5,6)
(Dia, 1,4,7) (Livus, 3,7,5)
(Alex, 2,5,8) (Alain, 9,5,8)

A = LOAD 'data.log' AS (t1:tuple(name:chararray, t1a:int,
t1b:int,t1c:int),t2:tuple(name:chararray,
t2a:int,t2b:int,t2c:int));

DUMP A;

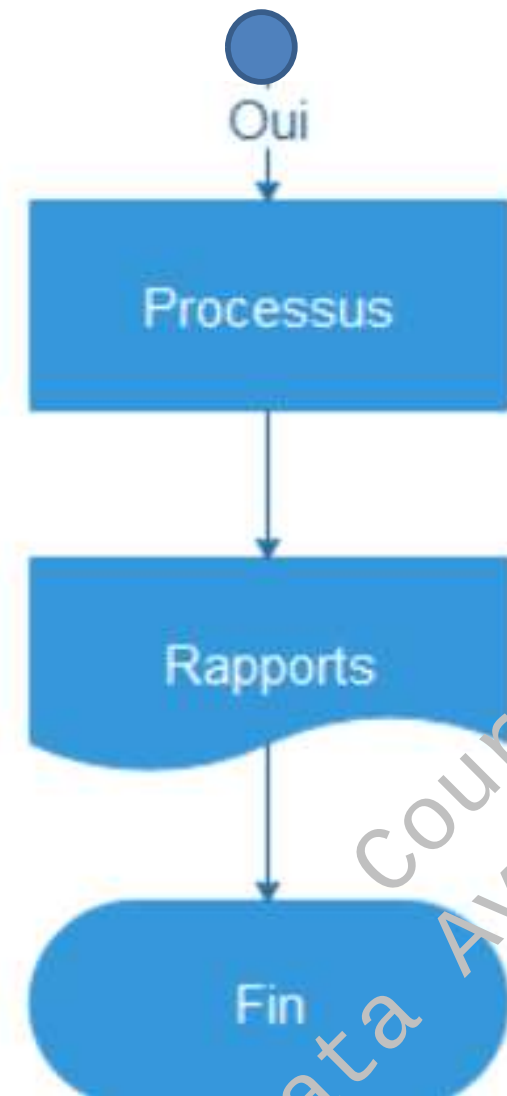
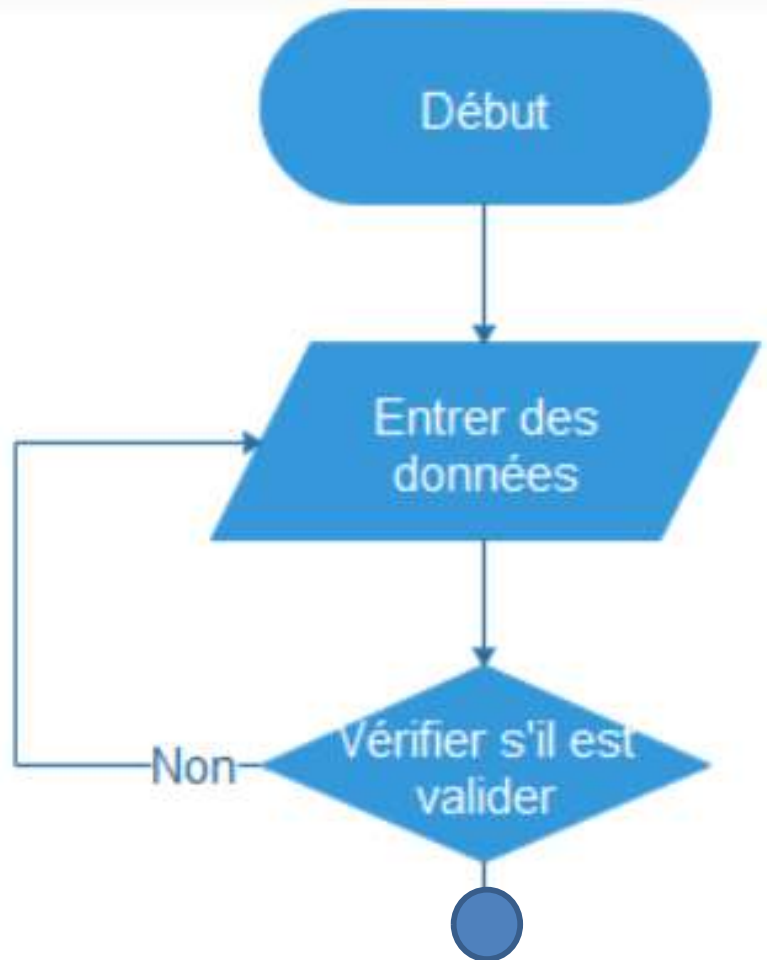
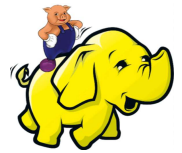
((jim, 6, 8,9), (Rach, 7,5,6))
((Dia, 1,4,7), (Livus, 3,7,5))
((Alex, 2,5,8), (Alain, 9,5,8))

X = FOREACH A GENERATE t1.name, t1.t1a,t2.$0;

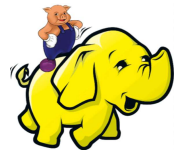
DUMP X;

(Jim, 6,7)
(Dia, 1,3)
(Alex,2,9)
```

Annexe : Logigramme (organigramme)



Annexe : Exemple d'UDF (Pig Latin)



- Exemple UDF : fonction qui calcule la longueur de chaque mot dans un jeu de données.

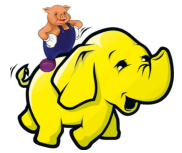
```
REGISTER myudfs.jar;  
  
DEFINE WordLengths org.myudfs.WordLengthUDF();  
  
input = LOAD 'input.txt' AS (line: chararray);  
words = FOREACH input GENERATE FLATTEN(TOKENIZE(line)) as word;  
word_lengths = FOREACH words GENERATE word, WordLengths(word) as  
word_length;  
  
DUMP word_lengths;
```

Pour plus d'infos sur les UDF pig voir :

[User Defined Functions \(apache.org\)](https://pig.apache.org/docs/r0.17.0/udf.html)

<https://pig.apache.org/docs/r0.17.0/udf.html>

Annexe : Exemple d'UDF (Python)



- UDF python
<https://pig.apache.org/docs/r0.17.0/udf.html#python-udfs>

```
register 'test.py' using org.apache.pig.scripting.streaming.python.PythonScriptEngine as myfuncs;
```

- Enregistrer un script python « test.py » comme une fonction Pig Latin « myfuncs »
- org.apache.pig.scripting.streaming.python.PythonScriptEngine → classe java qui exécute le script python en tant qu'une UDF Pig